基于 RISC-V 的高性能 CPU 设计及 FPGA 验证

——「七星微」企业命题·任务计划书

2026 年 3 月

目　录

1 项目概述 **4**

1.1 项目背景 4

1.2 项目目标 4

1.3 技术指标 4

1.4 开发平台 4

1.4.1 ALINX AX7020 开发板规格 4

2 CPU 微架构设计 **6**

2.1 总体架构 6

2.2 核心模块设计 6

2.2.1 取指阶段（IF Stage） 6

2.2.2 译码阶段（ID Stage） 6

2.2.3 执行阶段（EX Stage） 6

2.2.4 访存阶段（MEM Stage） 7

2.2.5 写回阶段（WB Stage） 7

2.2.6 CSR 模块 7

2.3 流水线冒险解决 7

2.3.1 数据冒险 — 全数据旁路转发（Forwarding） 7

2.3.2 控制冒险 — 分支预测 + 冲刷 7

2.3.3 结构冒险 — 资源分离 8

3 SoC 系统集成 **9**

3.1 SoC 总体架构 9

3.2 总线设计 9

3.3 外设模块 9

3.3.1 UART 控制器 9

3.3.2 Timer 9

3.3.3 GPIO 9

4 RTL 模块清单与文件规划 10

4.1 文件目录结构 10

4.2 模块信号接口表 11

4.2.1 CPU 顶层接口（qxw_cpu_top） 11

5 仿真验证计划 12

5.1 验证策略 12

5.2 模块级验证 12

5.3 系统级验证 12

5.4 仿真工具 12

6 FPGA 实现方案 13

6.1 目标器件 13

6.2 综合与实现流程 13

6.3 资源与时序预估 13

6.4 硬件调试 13

7 软件开发计划 14

7.1 交叉编译环境 14

7.2 启动代码 14

7.3 测试程序 14

7.4 CoreMark 基准测试 14

8 性能优化策略 15

8.1 优化技术清单 15

8.2 性能分析方法 15

8.3 优化迭代计划 15

9 项目进度规划 16

9.1 总体时间线（4 周） 16

9.2 详细里程碑 16

9.2.1 第一阶段：预研与架构设计（Day 1–3） 16

9.2.2 第二阶段：RTL 编码与模块级验证（Day 3–10） 16

9.2.3 第三阶段：系统集成与系统级仿真（Day 10–16） 16

9.2.4 第四阶段：FPGA 综合与板级调试（Day 16–20） 16

9.2.5 第五阶段：性能优化与基准测试（Day 20–24） 17

9.2.6 第六阶段：文档整理与成果验收（Day 24–28） 17

10 交付物清单 18

10.1 文档交付 18

10.2 代码交付 18
10.3 FPGA 原型系统 18

11 风险分析与应对 19

1 项目概述

1.1 项目背景

RISC-V 作为开源、精简且可扩展的指令集架构（ISA），已成为处理器设计与学术研究的核心载体。本项目响应「七星微」企业命题，旨在采用 Verilog 硬件描述语言，独立设计一款基于 RISC-V 的高性能五级流水线 CPU，并在 FPGA 平台上完成原型验证与应用演示。

1.2 项目目标

1. 设计并实现一款功能完整的 **RV32IM** CPU，支持整数运算、乘除法运算、数据传输、控制转移等核心指令；
2. 实现经典**五级流水线架构**（取指、译码、执行、访存、写回），完善解决数据冒险、控制冒险和结构冒险；
3. 引入至少 **2 种高性能优化技术**：全数据旁路转发（Forwarding）和两位饱和计数器分支预测（BHT）；
4. 完成从 Verilog 建模、仿真验证到 **FPGA 综合实现**的全流程开发；
5. 在 FPGA 上运行 **CoreMark** 基准测试与应用程序，进行性能量化分析。

1.3 技术指标

指标项	目标值
指令集架构	RV32I + M 扩展 + Zicsr
流水线级数	5 级（IF/ID/EX/MEM/WB）
目标时钟频率	⩾ 50 MHz（目标 100 MHz）
CoreMark 性能	⩾ 250 iterations/s @ 100 MHz
FPGA 资源占用	< 5,000 LUTs
FPGA 开发板	ALINX AX7020（Zynq-7020）
开发工具	Xilinx Vivado 2020.2+

1.4 开发平台

1.4.1 ALINX AX7020 开发板规格

参数	规格
FPGA 芯片	Xilinx Zynq-7020（XC7Z020CLG400-2）
逻辑单元	85K（53,200 LUTs / 106,400 FFs）
Block RAM	4.9 Mb（140 × 36Kb）
DSP48E1	220 个
DDR3 SDRAM	512 MB（PS 端）

参数	规格
Flash	QSPI 256 Mb
时钟源	50 MHz 有源晶振
串口	USB-UART（CP2102/CH340）
用户 LED	4 个
用户按键	4 个
拨码开关	4 位
扩展接口	2 × 40-pin 扩展口

2 CPU 微架构设计

2.1 总体架构

本 CPU 采用经典五级流水线架构，指令集选择 **RV32IM**（RV32I 基础整数指令集 + M 标准扩展），支持 32 位数据位宽、32 个通用寄存器。

流水线五个阶段为：

- **IF（取指）**：根据 PC 从指令存储器读取指令，包含分支预测逻辑；
- **ID（译码）**：解析指令字段，读取寄存器堆，生成控制信号与立即数；
- **EX（执行）**：ALU 运算、分支条件判断、地址计算、乘除法运算；
- **MEM（访存）**：执行 Load/Store 操作，数据对齐与符号扩展；
- **WB（写回）**：将运算结果或访存数据写回寄存器堆。

各阶段之间通过**级间寄存器**（Pipeline Register）隔离，实现指令重叠并行执行。

2.2 核心模块设计

2.2.1 取指阶段（IF Stage）

- **PC 寄存器**：维护当前取指地址，支持顺序递增（PC+4）、分支跳转目标和异常入口地址三路选择；
- **分支预测器**：采用两位饱和计数器（Bimodal Predictor），256 表项的分支历史表（BHT），通过 PC 低位索引，在取指阶段提前预测分支方向，减少控制冒险带来的流水线气泡；
- **指令存储器接口**：连接 BRAM 实现的指令存储器，单周期读取。

2.2.2 译码阶段（ID Stage）

- **指令译码器**：解析 opcode、funct3、funct7 字段，生成 ALU 操作类型、访存控制、寄存器写使能等控制信号；
- **立即数生成器**：支持 I/S/B/U/J 五种立即数格式的提取与符号扩展；
- **寄存器堆**：32 × 32-bit 通用寄存器，双读口单写口，同步写入、组合读出，x0 恒为零；
- **控制信号生成**：产出 ALU 选择、分支类型、访存读写、写回选择等流水线控制信号。

2.2.3 执行阶段（EX Stage）

- **ALU**：支持 RV32I 全部算术逻辑运算（ADD/SUB/AND/OR/XOR/SLL/SRL/SRA/SLT/SLTU）；
- **乘除法单元**（M 扩展）：支持 MUL/MULH/MULHSU/MULHU/DIV/DIVU/REM/REMU 八条指令；乘法采用单周期实现（利用 FPGA DSP48E1 资源），除法采用迭代实现（32 周期）；
- **分支判断**：比较 rs1/rs2 计算分支条件（BEQ/BNE/BLT/BGE/BLTU/BGEU），与预测结果对比，预测错误时产生冲刷信号；
- **地址计算**：计算分支跳转目标地址、Load/Store 访存地址。

2.2.4 访存阶段（MEM Stage）

- **数据存储器接口**：连接 BRAM 实现的数据存储器，支持字节/半字/字访问；
- **数据对齐**：Store 指令的字节/半字写掩码生成；
- **数据提取**：Load 指令的字节/半字选择与符号扩展/零扩展（LB/LH/LW/LBU/LHU）。

2.2.5 写回阶段（WB Stage）

- **结果选择器**：从 ALU 结果、访存数据、PC+4（JAL/JALR）、CSR 读取值中选择写回数据；
- **寄存器写回**：将选定结果写入目标寄存器。

2.2.6 CSR 模块

支持机器模式（M-mode）基础 CSR 寄存器：

CSR 寄存器	地址	功能
mstatus	0x300	机器状态寄存器
mtvec	0x305	异常入口地址
mepc	0x341	异常返回地址
mcause	0x342	异常原因
mcycle	0xB00	周期计数器
minstret	0xB02	指令计数器
mvendorid	0xF11	厂商 ID
marchid	0xF12	架构 ID

支持 CSRRW/CSRRS/CSRRC 三条 CSR 读写指令及其立即数变体。

2.3 流水线冒险解决

2.3.1 数据冒险 — 全数据旁路转发（Forwarding）

设置独立的**转发单元**（Forwarding Unit），检测 EX/MEM/WB 阶段结果寄存器与 ID 阶段源寄存器的 RAW 依赖，将最新结果旁路至 ID/EX 阶段输入端，优先级为 EX > MEM > WB。

Load-Use 冒险：当 EX 阶段为 Load 指令且其目的寄存器被下一条指令使用时，插入一个气泡（stall 一周期），等待 MEM 阶段数据就绪后通过转发解决。

2.3.2 控制冒险 — 分支预测 + 冲刷

- **分支预测**：取指阶段通过两位饱和计数器 BHT 预测分支方向，预测跳转时直接修改 PC 为预测目标地址；
- **预测验证**：执行阶段计算实际分支结果，与预测对比；
- **预测错误恢复**：预测错误时冲刷 IF/ID 阶段的错误指令，更新 BHT 表项，将 PC 修正为正确地址；

- **预计准确率**：循环密集型程序中预测准确率可达 85%–95%。

2.3.3 结构冒险 — 资源分离

采用 **Harvard 架构**，指令存储器和数据存储器使用独立的 BRAM 端口，取指和访存可同时进行，彻底避免存储器结构冒险。

3 SoC 系统集成

3.1 SoC 总体架构

CPU 核心通过一个**简易同步总线**连接片上存储器和外设，构成最小 SoC 系统：

组件	地址范围	大小	实现方式
指令 ROM	0x0000_0000 – 0x0000_3FFF	16 KB	BRAM
数据 RAM	0x0001_0000 – 0x0001_3FFF	16 KB	BRAM
UART	0x1000_0000 – 0x1000_00FF	256 B	简易 UART TX/RX
Timer	0x1000_1000 – 0x1000_10FF	256 B	64-bit mtime 计时器
GPIO	0x1000_2000 – 0x1000_20FF	256 B	LED/按键控制

3.2 总线设计

采用简易同步总线协议（非 AXI4），减少面积开销：

- 信号：`addr[31:0]`、`wdata[31:0]`、`rdata[31:0]`、`wen`、`ren`、`wmask[3:0]`、`ready`
- 单周期访问 BRAM，外设通过地址译码选通
- 后续可升级为 AXI4-Lite 接口

3.3 外设模块

3.3.1 UART 控制器

- 波特率：115200 bps（可配置）
- 支持发送/接收，8N1 格式
- 用于 CoreMark 结果输出和调试信息打印

3.3.2 Timer

- 64-bit 自由运行计数器（`mtime`），以系统时钟递增
- 用于 CoreMark 计时和性能统计

3.3.3 GPIO

- 4-bit LED 输出寄存器
- 4-bit 按键/拨码开关输入寄存器
- 用于基础 I/O 演示

4 RTL 模块清单与文件规划

4.1 文件目录结构

```
qixingwei/
├── rtl/
│   ├── core/                   # CPU 核心模块
│   │   ├── qxw_defines.vh      # 全局宏定义与参数
│   │   ├── qxw_pc_reg.v        # PC 寄存器
│   │   ├── qxw_if_stage.v      # 取指阶段
│   │   ├── qxw_id_stage.v      # 译码阶段
│   │   ├── qxw_ex_stage.v      # 执行阶段
│   │   ├── qxw_mem_stage.v     # 访存阶段
│   │   ├── qxw_wb_stage.v      # 写回阶段
│   │   ├── qxw_regfile.v       # 寄存器堆
│   │   ├── qxw_alu.v           # 算术逻辑单元
│   │   ├── qxw_muldiv.v        # 乘除法单元 (M 扩展)
│   │   ├── qxw_forwarding.v    # 数据转发单元
│   │   ├── qxw_hazard_ctrl.v   # 冒险检测与控制
│   │   ├── qxw_branch_pred.v   # 分支预测器
│   │   ├── qxw_csr.v           # CSR 寄存器模块
│   │   └── qxw_cpu_top.v       # CPU 核心顶层
│   ├── soc/                    # SoC 集成
│   │   ├── qxw_soc_top.v       # SoC 顶层
│   │   ├── qxw_bus.v           # 总线互联
│   │   ├── qxw_uart.v          # UART 控制器
│   │   ├── qxw_timer.v         # Timer 计时器
│   │   └── qxw_gpio.v          # GPIO 控制器
│   └── mem/                    # 存储器
│       ├── qxw_imem.v          # 指令存储器 (BRAM)
│       └── qxw_dmem.v          # 数据存储器 (BRAM)
├── tb/                         # 仿真测试
│   ├── tb_cpu_top.v            # CPU 系统级测试
│   ├── tb_alu.v                # ALU 模块测试
│   ├── tb_regfile.v            # 寄存器堆测试
│   ├── tb_branch_pred.v        # 分支预测器测试
│   └── riscv_tests/            # riscv-tests 测试程序
├── sim/                        # 仿真脚本
│   └── Makefile                # iverilog/Verilator 仿真
├── fpga/                       # FPGA 工程
```

```
│   ├── constraints/
│   │   └── ax7020.xdc          # AX7020 引脚约束
│   └── scripts/
│       └── build.tcl           # Vivado TCL 构建脚本
├── sw/                         # 软件
│   ├── startup/
│   │   └── start.S             # 启动代码
│   ├── lib/
│   │   ├── uart.c              # UART 驱动
│   │   └── timer.c             # Timer 驱动
│   ├── tests/                  # 测试程序
│   │   ├── basic_test.c        # 基础指令测试
│   │   ├── matrix_mul.c        # 矩阵乘法
│   │   └── sort.c              # 排序算法
│   ├── coremark/               # CoreMark 基准程序
│   │   └── ...
│   ├── linker.ld               # 链接脚本
│   └── Makefile                # 交叉编译
└── doc/                        # 文档
    ├── 设计报告.md
    ├── 验证报告.md
    └── 性能分析报告.md
```

4.2 模块信号接口表

4.2.1 CPU 顶层接口（qxw_cpu_top）

信号名	方向	位宽	说明
clk	input	1	系统时钟
rst_n	input	1	异步复位（低有效）
imem_addr	output	32	指令存储器地址
imem_rdata	input	32	指令存储器读数据
dmem_addr	output	32	数据存储器地址
dmem_wdata	output	32	数据存储器写数据
dmem_rdata	input	32	数据存储器读数据
dmem_wen	output	1	数据存储器写使能
dmem_wmask	output	4	数据存储器字节写掩码

5 仿真验证计划

5.1 验证策略

采用**自底向上**的分层验证策略：先完成模块级功能验证，再进行系统级集成验证。

5.2 模块级验证

模块	验证重点	测试用例数（估计）
ALU	全部 RV32I/M 运算类型、边界值、溢出	≥ 100
寄存器堆	读写冲突、x0 恒零、同时读写	≥ 30
分支预测器	预测准确率、状态转移正确性	≥ 50
译码器	全部指令类型解码正确性	≥ 47（RV32IM 全部指令）
转发单元	全部转发路径覆盖	≥ 20
冒险控制	load-use stall、分支冲刷	≥ 15

5.3 系统级验证

1. **riscv-tests 官方测试集**：运行 RV32I 和 RV32M 全部指令测试用例，验证指令执行正确性；
2. **自定义冒险测试**：构造 RAW、Load-Use、分支冒险等典型场景，验证转发和冲刷机制；
3. **CoreMark 仿真运行**：在仿真环境中运行 CoreMark，验证程序执行正确性并预估性能。

5.4 仿真工具

工具	用途
Icarus Verilog (iverilog)	快速功能仿真
GTKWave	波形查看与调试
Verilator（可选）	高速 cycle-accurate 仿真
Vivado Simulator	综合后时序仿真

6 FPGA 实现方案

6.1 目标器件

Xilinx Zynq-7020（XC7Z020CLG400-2），搭载于 ALINX AX7020 开发板。本项目仅使用 PL（Programmable Logic）端 FPGA 资源，不使用 PS（ARM 处理器）端。

6.2 综合与实现流程

1. **代码准备**：确保所有 Verilog 代码可综合，移除仿真专用语句；
2. **创建 Vivado 工程**：选择器件 XC7Z020CLG400-2，导入 RTL 源文件；
3. **约束配置**：
   - 时钟约束：50 MHz 输入时钟（板载晶振），PLL 倍频至 100 MHz；
   - 引脚约束：UART TX/RX、LED、按键、复位对应 AX7020 实际引脚；
4. **综合（Synthesis）**：使用 Vivado 综合引擎，优化策略选择面积优先（Area Explore）；
5. **实现（Implementation）**：布局布线，检查时序报告确保满足约束；
6. **比特流生成**：生成 `.bit` 文件用于 FPGA 配置。

6.3 资源与时序预估

资源类型	预估使用量	芯片总量	占比
LUT	3,000 – 4,500	53,200	6% – 8%
FF	1,500 – 2,500	106,400	1.5% – 2.5%
BRAM (36Kb)	8 – 16	140	6% – 11%
DSP48E1	4 – 8	220	2% – 4%

6.4 硬件调试

- 使用 Vivado **ILA**（Integrated Logic Analyzer）核抓取关键信号：PC、指令、流水线寄存器、控制信号；
- 通过 **UART** 串口输出调试信息与程序运行结果；
- LED 显示运行状态指示。

7 软件开发计划

7.1 交叉编译环境

- 编译器：`riscv32-unknown-elf-gcc`（GCC 13+）
- 编译选项：`-march=rv32im -mabi=ilp32 -O2 -nostdlib`
- 链接脚本：自定义 `linker.ld`，配置代码段（.text）和数据段（.data/.bss）地址

7.2 启动代码

启动文件 `start.S` 完成以下初始化：

1. 设置栈指针（sp）到数据 RAM 顶端；
2. 清零 BSS 段；
3. 跳转到 C 语言 `main()` 函数。

7.3 测试程序

程序	功能	目的
basic_test	基础指令逐条测试	验证指令正确性
matrix_mul	4×4 矩阵乘法	演示整数运算能力
sort	快速排序 / 冒泡排序	演示分支与访存性能
led_blink	LED 流水灯	FPGA I/O 演示

7.4 CoreMark 基准测试

- 目标：CoreMark/MHz ⩾ 2.5（即 100 MHz 下 ⩾ 250 iterations/s）
- 使用 Timer 模块提供精确计时
- 通过 UART 串口输出 CoreMark 评分结果

8 性能优化策略

8.1 优化技术清单

优化技术	分类	预期收益
全数据旁路转发	数据冒险	消除 90%+ RAW 停顿
两位饱和计数器分支预测	控制冒险	分支预测准确率 85%–95%
乘法器 DSP 映射	面积优化	乘法不占用 LUT
Harvard 架构	结构冒险	取指/访存零冲突

8.2 性能分析方法

1. **流水线停顿率**：通过 mcycle 和 minstret 计算 CPI（Cycles Per Instruction），目标 CPI < 1.5；
2. **分支预测准确率**：统计预测正确/错误次数，输出准确率报告；
3. **CoreMark/MHz**：标准基准评分，对比优化前后差异；
4. **资源占用**：Vivado 综合报告中提取 LUT/FF/BRAM/DSP 占用数据。

8.3 优化迭代计划

1. **基线版本**：无分支预测、有转发的五级流水线 → 测量基线 CPI 和 CoreMark；
2. **优化版本 1**：加入分支预测 → 对比 CPI 提升；
3. **优化版本 2**：调整 BHT 参数、优化关键路径时序 → 对比最终性能。

9 项目进度规划

9.1 总体时间线（4 周）

阶段	时间	主要任务	交付物
第一阶段	第 1 周（Day 1–3）	预研与架构设计	ISA 功能清单、架构设计文档、开发环境搭建
第二阶段	第 1–2 周（Day 3–10）	RTL 编码与模块级验证	各模块 Verilog 代码、模块级仿真报告
第三阶段	第 2–3 周（Day 10–16）	系统集成与系统级仿真	完整 CPU 系统、riscv-tests 通过报告
第四阶段	第 3 周（Day 16–20）	FPGA 综合与板级调试	Vivado 工程、比特流文件、板级调试日志
第五阶段	第 3–4 周（Day 20–24）	性能优化与基准测试	CoreMark 评分、性能分析报告
第六阶段	第 4 周（Day 24–28）	文档整理与成果验收	设计报告、验证报告、演示视频、答辩 PPT

9.2 详细里程碑

9.2.1 第一阶段：预研与架构设计（Day 1–3）

- ☐ 精读 RISC-V Unprivileged ISA 手册，输出 RV32IM 指令集功能覆盖清单
- ☐ 完成五级流水线架构详细设计，绘制架构图与模块接口图
- ☐ 搭建开发环境（Vivado + iverilog + riscv-gcc + GTKWave）
- ☐ 编写 ALU、寄存器堆等简单模块验证工具链可用性
- ☐ 制定编码规范与验证计划

9.2.2 第二阶段：RTL 编码与模块级验证（Day 3–10）

- ☐ 完成全部核心模块 Verilog 编码（IF/ID/EX/MEM/WB + 寄存器堆 + ALU + M 扩展）
- ☐ 完成转发单元、冒险控制、分支预测器编码
- ☐ 编写全部模块级 Testbench，覆盖边界情况与异常场景
- ☐ 完成模块级仿真，修复逻辑缺陷，输出模块级验证报告

9.2.3 第三阶段：系统集成与系统级仿真（Day 10–16）

- ☐ 集成 CPU 核心与 SoC 外设，搭建完整系统
- ☐ 运行 riscv-tests 官方测试集，确保全部通过
- ☐ 运行自定义冒险测试程序，验证流水线正确性
- ☐ 在仿真环境运行 CoreMark，验证功能正确

9.2.4 第四阶段：FPGA 综合与板级调试（Day 16–20）

- ☐ 创建 Vivado 工程，完成综合与布局布线
- ☐ 编写 AX7020 XDC 约束文件
- ☐ 下载比特流到开发板，完成基础功能验证
- ☐ 使用 ILA 调试关键信号，修复时序问题

9.2.5 第五阶段：性能优化与基准测试（Day 20–24）

- ☐ 在 FPGA 上运行 CoreMark，测量实际性能
- ☐ 对比优化前后 CPI、分支预测准确率
- ☐ 运行矩阵乘法、排序等应用程序
- ☐ 统计 FPGA 资源占用，分析面积-性能平衡

9.2.6 第六阶段：文档整理与成果验收（Day 24–28）

- ☐ 撰写完整设计报告（架构说明、模块设计、接口规范）
- ☐ 撰写验证报告（仿真结果、FPGA 测试数据）
- ☐ 撰写性能分析报告（CoreMark 评分、资源占用、优化对比）
- ☐ 录制功能演示视频
- ☐ 准备答辩 PPT

10 交付物清单

10.1 文档交付

文档名称	格式	说明
CPU 架构设计说明书	Word + PDF	含架构图、数据流图、控制信号时序图
Verilog 建模规范	Word + PDF	编码规范、模块命名规则
验证计划与测试报告	Word + PDF	模块级/系统级仿真结果、波形截图
性能分析报告	Word + PDF	CoreMark 评分、CPI、资源占用、优化对比
FPGA 适配指南	Word + PDF	Vivado 工程配置、约束文件、调试方法

10.2 代码交付

内容	说明
RTL 源代码	完整 Verilog 设计代码，注释率 $\geqslant$ 30%
Testbench 代码	模块级与系统级测试基准
FPGA 工程	Vivado 工程文件、约束文件、比特流
软件代码	启动代码、驱动库、测试程序、CoreMark
构建脚本	Makefile、TCL 脚本

10.3 FPGA 原型系统

- 可稳定运行的 FPGA CPU 原型（AX7020 开发板）
- 支持 RV32IM 指令集
- 时钟频率 $\geqslant$ 50 MHz
- UART 串口输出调试信息与 CoreMark 结果
- LED 状态指示

11 风险分析与应对

风险	等级	应对措施
时序不收敛（关键路径过长）	高	寄存器重定时、拆分组合逻辑、降低目标频率
LUT 超 5K 预算	中	简化除法器（软件实现）、减小 BHT 表项、共享资源
分支预测效果不佳	低	调整 BHT 大小/索引策略、可选升级 GShare
CoreMark 性能不达标	中	优化编译选项、增加 I-Cache、调整流水线结构
FPGA 板级调试困难	中	充分仿真验证、善用 ILA、UART 调试打印